

МФТИ

Алгоритмы и структуры данных II

Семинар 04

Базовые алгоритмы на графах (1)

Весна 2026

Строгие подробные решения с доказательствами

Содержание

Обозначения и соглашения	2
1 Задача 1. Экспоненциальный DFS без меток	3
2 Задача 2. Одновершинное дерево в лесу DFS	4
3 Задача 3. Сильная связность как отношение эквивалентности	5
4 Задача 4. Запросы о предках	6
5 Задача 5. Гамильтонов путь в турнире	7
6 Задача 6. Критерии двудольности	8
7 Задача 7. Число путей в DAG	9
8 Задача 8. Система равенств и строгих неравенств	10
9 Задача 9. Цепная реакция бомб на прямой	11
10 Задача 10. Приведение перестановки к тождественной	12
11 Задача 11. Вершины, из которых достижимы все остальные	13
12 Задача 12. Длины всех циклов кратны k	14
13 Задача 13. Компоненты связности дополнения графа	15

Обозначения и соглашения

Во всех оценках n обозначает размер входа, если в условии конкретной задачи не сказано иначе. Значение $+\infty$ используется для недостижимого состояния в задачах минимизации, а $-\infty$ — в задачах максимизации. Слово «путь» может означать маршрут с повторениями вершин только там, где это прямо следует из постановки; во всех остальных случаях путь простой.

1. Задача 1. Экспоненциальный DFS без меток

Решение. Рассмотрим ориентированный ациклический граф на вершинах $v_0, v_1, \dots, v_k$, в котором для каждого $i \leq k - 2$ есть рёбра

$$v_i \rightarrow v_{i+1}, \quad v_i \rightarrow v_{i+2}.$$

Запустим рекурсивную процедуру из v_0 , которая для каждой вершины рекурсивно вызывает себя на всех исходящих соседях, но не запоминает уже обработанные вершины.

Пусть $T(i)$ — число вызовов, порождённых вызовом из v_i . Тогда

$$T(i) \geq T(i+1) + T(i+2), \quad T(k) = 1, \quad T(k-1) \geq 1.$$

Следовательно, $T(0)$ не меньше числа Фибоначчи F_k , а

$$F_k = \Theta(\varphi^k), \quad \varphi = \frac{1 + \sqrt{5}}{2}.$$

При этом в графе $k + 1$ вершина и менее $2k$ рёбер. Причина взрыва времени — повторный обход одной вершины по каждому различному пути, ведущему к ней.

Доказательство корректности. Граф ациклический, поскольку номер вершины строго возрастает вдоль каждого ребра. Из v_i процедура обязана отдельно выполнить рекурсивные вызовы для v_{i+1} и v_{i+2} ; отсутствие меток не позволяет переиспользовать результат. Поэтому указанная рекуррентная нижняя граница точна с точностью до добавочной константы и даёт экспоненциальное число вызовов на линейном по размеру семействе DAG. $\square$

Сложность. Обычный DFS с метками работает за $\mathcal{O}(n + m)$, а описанный вариант на семействе выше — за $\Omega(\varphi^n)$.

2. Задача 2. Одновершинное дерево в лесу DFS

Решение. Да, может. Возьмём три вершины a, v, b и рёбра

$$v \rightarrow a, \quad b \rightarrow v.$$

Пусть внешний цикл DFS просматривает вершины в порядке a, v, b . Сначала a становится одноэлементным деревом. Затем запускается DFS из v ; его единственный исходящий сосед a уже посещён, поэтому дерево с корнем v содержит только v . Наконец, при запуске из b вершина v также уже посещена.

Таким образом, у v есть и входящее, и исходящее ребро, но оба не порождают дочернюю вершину в её дереве DFS. Входящие рёбра вообще не просматриваются из v , а исходящее ведёт в уже завершённое дерево.

3. Задача 3. Сильная связность как отношение эквивалентности

Решение. Отношение $u \sim v$ означает взаимную достижимость.

- a) **Рефлексивность.** Путь длины 0, состоящий из одной вершины v , прост и ведёт из v в v . Поэтому $v \sim v$.
- b) **Симметричность.** В определении $u \sim v$ одновременно требуются пути $u \rightsquigarrow v$ и $v \rightsquigarrow u$. Перестановка этих двух требований даёт $v \sim u$.
- c) **Транзитивность.** Из $u \sim v$ и $v \sim w$ получаем ориентированные маршруты $u \rightsquigarrow v \rightsquigarrow w$ и $w \rightsquigarrow v \rightsquigarrow u$. Возможно, в конкатенации повторяются вершины. Из любого ориентированного маршрута можно удалять участок между двумя последовательными появлениями одной вершины; начало и конец сохраняются, а длина уменьшается. После конечного числа удалений получится простой путь. Значит, существуют простые пути $u \rightsquigarrow w$ и $w \rightsquigarrow u$, то есть $u \sim w$.

Теперь пример отсутствия двух внутренне непересекающихся путей. Возьмём вершины u, x, v и рёбра

$$u \rightarrow x, \quad x \rightarrow v, \quad v \rightarrow x, \quad x \rightarrow u.$$

Вершины u и v взаимно достижимы. Однако любой путь из u в v и любой путь из v в u обязан проходить через x , поскольку после выхода из u и из v единственный следующий выбор — x . Следовательно, два пути имеют общий внутренний узел x .

Доказательство корректности. Три свойства доказаны непосредственно из определения и леммы об удалении циклических участков из маршрута. Пример действительно удовлетворяет взаимной достижимости и одновременно содержит разделяющую вершину x ; поэтому он отвечает обоим требованиям. $\square$

4. Задача 4. Запросы о предках

Решение. Выполним DFS из корня. При входе в вершину v запишем время $tin[v]$, а после полного обхода её поддерева — $tout[v]$. Таймер увеличивается при каждом событии.

Тогда

$$u \text{ является предком } v \iff tin[u] \leq tin[v] \text{ и } tout[v] \leq tout[u]. \quad (1)$$

При строгом понимании слова «предок» дополнительно проверяется $u \neq v$.

Доказательство корректности. Рекурсивный вызов DFS для u начинается в момент $tin[u]$ и заканчивается в $tout[u]$. Все и только вершины поддерева u обрабатываются целиком внутри этого временного интервала: рекурсивный обход не покидает поддерево до возврата из u , а к вершинам вне поддерева переходит лишь раньше или позже. Поэтому временной интервал v вложен в интервал u ровно тогда, когда v находится в поддереве u , что и выражает (1). $\square$

Сложность. Предобработка $\mathcal{O}(n)$, каждый запрос $\mathcal{O}(1)$; всего $\mathcal{O}(n + q)$ времени и $\mathcal{O}(n)$ памяти.

5. Задача 5. Гамильтонов путь в турнире

Решение. Будем поддерживать ориентированный путь

$$p_1 \rightarrow p_2 \rightarrow \cdots \rightarrow p_k$$

по уже добавленным вершинам. Для новой вершины v найдём первый индекс i , для которого $v \rightarrow p_i$.

Если такой индекс есть, то для всех $j < i$, в частности для $i - 1$, верно $p_j \rightarrow v$: между двумя вершинами турнира есть ровно одна дуга, а $v \rightarrow p_j$ не выполнилось. Поэтому вставляем v перед p_i :

$$\cdots \rightarrow p_{i-1} \rightarrow v \rightarrow p_i \rightarrow \cdots .$$

При $i = 1$ ставим v в начало. Если подходящего i нет, все $p_j \rightarrow v$, и v ставится в конец.

Доказательство корректности. Индукция по числу добавленных вершин. Путь из одной вершины корректен. При вставке новой вершины все старые соседние пары сохраняются, а две новые дуги имеют правильное направление по выбору первого i . В случае добавления в конец правильна последняя дуга. Значит, после каждого шага получается ориентированный путь, содержащий каждую обработанную вершину ровно один раз. После n шагов он гамильтонов. $\square$

Сложность. Одна вставка требует $\mathcal{O}(k)$ проверок, всего $\mathcal{O}(n^2)$. Турнир имеет $\Theta(n^2)$ рёбер, поэтому это линейное время от размера явно заданного графа.

6. Задача 6. Критерии двудольности

Решение. Эквивалентность определения и разбиения на доли. Если $\chi(G) \leq 2$, положим в одну долю вершины первого цвета, в другую — второго; ребро не может иметь концы одного цвета. Обратно, раскрасим две доли двумя цветами. Поскольку рёбра идут только между долями, раскраска правильна.

Из двудольности следует отсутствие нечётных циклов. При движении по ребру доля меняется. После нечётного числа рёбер мы оказываемся в другой доле, поэтому вернуться в исходную вершину невозможно. Значит, каждый цикл имеет чётную длину.

Из отсутствия нечётных циклов следует двудольность. В каждой компоненте выберем корень и запустим BFS, присвоив вершине цвет $\text{depth}[v] \bmod 2$. Предположим, что ребро $\{u, v\}$ соединило одинаковые цвета. Пути дерева BFS от корня до u и v имеют глубины одной чётности. Пусть z — их последняя общая вершина. Тогда дерево содержит внутренне непересекающиеся пути $z \rightsquigarrow u$ и $z \rightsquigarrow v$, а вместе с ребром uv они образуют цикл длины

$$(\text{depth}[u] - \text{depth}[z]) + (\text{depth}[v] - \text{depth}[z]) + 1,$$

которая нечётна. Противоречие. Следовательно, раскраска правильна.

Дерево. В дереве циклов нет, в частности нет нечётных циклов, а значит, оно двудольно. Непосредственно его доли задаются чётностью расстояния от произвольного корня.

Доказательство корректности. Каждый переход между формулировками доказан в обе стороны. В конструкции BFS конфликт цветов неизбежно создаёт простой нечётный цикл через последнего общего предка, поэтому при отсутствии такого цикла конфликт невозможен. Это завершает доказательство всех эквивалентностей и следствия для дерева. $\square$

7. Задача 7. Число путей в DAG

Решение. Будем считать непустые ориентированные пути между произвольными парами вершин; пути с одинаковой последовательностью вершин считаются один раз. Получим топологический порядок. В обратном порядке вычислим

$$dp[v] = 1 + \sum_{(v,u) \in E} dp[u]. \quad (2)$$

Здесь 1 соответствует пути длины 0, состоящему только из v . Тогда число непустых путей равно

$$\sum_{v \in V} (dp[v] - 1). \quad (3)$$

Если нужно считать пути из фиксированного s , берётся $dp[s] - 1$; если в фиксированный t , используется симметричная динамика по входящим рёбрам.

Доказательство корректности. Любой непустой путь, начинающийся в v , имеет единственное первое ребро (v, u) , после которого идёт произвольный путь, начинающийся в u , включая нулевой. Разные первые рёбра задают непересекающиеся классы, поэтому сумма (2) точна. Обратный топологический порядок гарантирует, что все $dp[u]$ уже известны. Наконец, каждый непустой путь имеет единственную первую вершину, так что сумма (3) не создаёт повторов. $\square$

Сложность. Топологическая сортировка и все переходы занимают $\mathcal{O}(n+m)$ времени; память $\mathcal{O}(n+m)$ вместе с графом.

8. Задача 8. Система равенств и строгих неравенств

Решение. Сначала системой непересекающихся множеств объединим i, j для каждого условия $x_i = x_j$. После этого каждое множество должно получить одно общее значение.

Для ограничения $x_i < x_j$:

- если i, j попали в одну компоненту равенств, система противоречива;
- иначе добавим дугу $[i] \rightarrow [j]$ между компонентами.

Система имеет решение тогда и только тогда, когда полученный граф строгих неравенств ацикличесен.

Если граф является DAG, возьмём любой топологический порядок и присвоим компоненте её номер в этом порядке (либо длину максимального пути до неё). Все переменные компоненты получают это значение.

Доказательство корректности. Если в графе есть цикл

$$C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_q \rightarrow C_1,$$

ограничения требуют $x_{C_1} < x_{C_2} < \dots < x_{C_q} < x_{C_1}$, что невозможно. Строгое условие внутри компоненты равенства даёт такое же немедленное противоречие.

Обратно, в топологическом порядке начало каждой дуги расположено раньше конца. Поэтому присвоенные целые номера строго возрастают вдоль всех дуг и удовлетворяют каждому неравенству; объединённые переменные имеют одинаковое значение. Построено явное решение. $\square$

Сложность. Для q условий: $\mathcal{O}((n+q)\alpha(n))$ на объединения и $\mathcal{O}(n+q)$ на проверку DAG; память $\mathcal{O}(n+q)$.

9. Задача 9. Цепная реакция бомб на прямой

Решение. Отсортируем бомбы по координате. Для каждой бомбы i двумя бинарными поисками найдём непрерывный диапазон индексов

$$[L_i, R_i] = \{j : x_i - r_i \leq x_j \leq x_i + r_i\}.$$

Построим неявный ориентированный граф: из i есть дуги ко всем вершинам этого диапазона. Если поджечь вершину, взорвутся ровно все достижимые из неё вершины. Минимальное число начальных поджогов равно числу компонент сильной связности с нулевой входящей степенью в конденсации.

Чтобы не строить $\Theta(n^2)$ дуг, используем дерево отрезков:

- его листья соответствуют бомбам;
- из каждой внутренней вершины идут дуги к двум детям;
- из листа i проводятся дуги в $\mathcal{O}(\log n)$ канонических вершин дерева, чьи отрезки разбивают $[L_i, R_i]$.

Тогда путь от i через каноническую вершину и нисходящие рёбра достигает ровно всех листьев требуемого диапазона.

Запускаем поиск SCC в расширенном графе. Технически внутренние вершины, которые не достижимы ни из одного листа через диапазонную дугу, не являются реальными состояниями; их игнорируем. Это можно сделать, предварительно пометив все вершины расширенного графа, достижимые из множества листьев, и считая входящие степени конденсации только на помеченном подграфе. Число SCC с нулевой входящей степенью, содержащих хотя бы один лист, есть ответ. По представителю такого SCC можно восстановить набор поджигаемых бомб.

Доказательство корректности. Сначала докажем эквивалентность графов. Каждая исходная дуга $i \rightarrow j$, где $j \in [L_i, R_i]$, заменена путём из листа i к каноническому отрезку и затем вниз к листу j . Обратно, любой путь от листа через первую диапазонную дугу может спуститься только к листу, лежащему в выбранном подотрезке, то есть не создаёт ложного непосредственного поражения. Последовательности таких переходов в точности моделируют цепные реакции.

В конденсации SCC получается DAG. Каждую его компоненту с нулевой входящей степенью необходимо активировать вручную: из другой компоненты она недостижима. Одной вершины из каждой такой компоненты достаточно, потому что внутри SCC все вершины взаимно достижимы, а из всех источников DAG достижимы все его вершины. Помеченные внутренние узлы дерева, не содержащие листьев, не могут создать дополнительный источник: раз они помечены, к ним ведёт путь от некоторого листа; первый вход такого пути либо помещает их в ту же SCC, либо даёт входящую дугу. Поэтому подсчёт листовых источников совпадает с источниками исходной конденсации бомб. $\square$

Сложность. Бинарные поиски занимают $\mathcal{O}(n \log n)$. Расширенный граф имеет $\mathcal{O}(n)$ вершин и $\mathcal{O}(n \log n)$ рёбер. SCC и конденсация строятся за линейное от него время. Итого $\mathcal{O}(n \log n)$ времени и $\mathcal{O}(n \log n)$ памяти.

10. Задача 10. Приведение перестановки к тождественной

Пункт Обмен любых двух элементов

Решение. Разложим перестановку на непересекающиеся циклы. Цикл длины ℓ можно исправить за $\ell - 1$ обменов: фиксируем одну позицию и последовательно ставим на место остальные элементы цикла. Поэтому, если циклов c ,

$$\text{ans} = \sum (\ell - 1) = n - c. \quad (4)$$

Ни один обмен не может увеличить число циклов перестановки более чем на один, а у тождественной перестановки n циклов. Из исходных c циклов нужно получить n , поэтому меньше $n - c$ обменов невозможно.

Пункт Обмен p_i и p_{p_i}

Решение. Операция всегда действует внутри одного цикла отображения $i \mapsto p_i$. Пусть фрагмент цикла имеет вид

$$i \mapsto j = p_i \mapsto q = p_j.$$

После обмена значений в позициях i и j получаем

$$p_j = j, \quad p_i = q.$$

То есть вершина j становится неподвижной, а из исходного цикла удаляется. Повторив операцию $\ell - 1$ раз, исправляем цикл длины ℓ .

Одна разрешённая операция не затрагивает два разных цикла и увеличивает общее число циклов ровно на один, пока выбранная позиция не фиксирована. Поэтому нижняя граница $n - c$ сохраняется, и ответ снова равен

$$\boxed{n - c}.$$

Доказательство корректности. Для обоих наборов операций дана конструкция из $n - c$ шагов и независимая нижняя граница через число циклов. Во втором пункте явное вычисление новых образов показывает, что каждый шаг действительно отделяет одну неподвижную точку и не может исправить более одной требуемой единицы дефицита числа циклов. Поэтому границы совпадают. $\square$

Сложность. Разложение перестановки на циклы выполняется за $\mathcal{O}(n)$ времени и $\mathcal{O}(n)$ памяти.

11. Задача 11. Вершины, из которых достижимы все остальные

Решение. Найдём компоненты сильной связности и построим конденсацию D . Это DAG. Утверждается, что ответ существует тогда и только тогда, когда в D ровно один источник (вершина входной степени 0). В этом случае ответом являются все исходные вершины единственной исходной SCC.

Доказательство корректности. Если компонента C достигает все остальные компоненты DAG, она обязана быть источником. Иначе существует дуга $B \rightarrow C$; путь из C в B вместе с этой дугой создал бы цикл в конденсации. Кроме того, другой источник не мог бы быть достижим из C . Значит, достигающая всех компонента является единственным источником.

Обратно, в любом конечном DAG каждая вершина достижима из некоторого источника: достаточно идти по входящим рёбрам назад, пока они не закончатся. Если источник единственный, каждая вершина достижима именно из него. Внутри SCC все вершины взаимно достижимы, поэтому каждая вершина исходной компоненты достигает весь исходный граф, а вершины других компонент — нет. $\square$

Сложность. Алгоритм Косарайю или Тарьяна, а также подсчёт степеней конденсации: $\mathcal{O}(n + m)$ времени и памяти.

12. Задача 12. Длины всех циклов кратны k

Решение. Попробуем присвоить каждой вершине остаток $d[v] \in \mathbb{Z}/k\mathbb{Z}$ так, чтобы для каждой дуги $u \rightarrow v$

$$d[v] \equiv d[u] + 1 \pmod{k}. \quad (5)$$

Полагаем $d[r] = 0$ для произвольного корня и обходим сильно связный граф. При первом посещении v назначаем требуемый остаток, а для каждой дуги к уже назначенной вершине проверяем (5). Все циклы имеют длину, кратную k , ровно тогда, когда противоречий не найдено.

Доказательство корректности. Если разметка существует, сложим равенства (5) вдоль цикла длины ℓ . Начальный остаток должен совпасть с конечным, поэтому $\ell \equiv 0 \pmod{k}$.

Пусть теперь длина каждого простого цикла кратна k . Любой замкнутый ориентированный маршрут раскладывается удалением повторяющихся участков на простые ориентированные циклы, поэтому его длина также кратна k . Для вершины v определим $d[v]$ как длину произвольного пути $r \rightsquigarrow v$ по модулю k . Определение корректно: для двух таких путей P, Q сильная связность даёт путь $R : v \rightsquigarrow r$, а замкнутые маршруты PR и QR имеют длины $0 \pmod{k}$; следовательно, $|P| \equiv |Q| \pmod{k}$. Продление пути на дугу $u \rightarrow v$ даёт (5). Значит, при выполнении условия обход не обнаружит конфликта. $\square$

Сложность. Каждая вершина и дуга обрабатывается постоянное число раз: $\mathcal{O}(n+m)$ времени, $\mathcal{O}(n)$ дополнительной памяти.

13. Задача 13. Компоненты связности дополнения графа

Решение. Храним множество U ещё не включённых в компоненты вершин в двусвязном списке, поддерживающем удаление за $\mathcal{O}(1)$. Начинаем новую BFS-компоненту с произвольной вершины из U .

При извлечении вершины v из очереди:

1. помечаем всех соседей v в исходном графе G ;
2. одним проходом по текущему списку U удаляем каждую *непомеченную* вершину u , добавляем её в очередь и в текущую компоненту;
3. снимаем метки с соседей v .

Непомеченная вершина не смежна с v в G , а значит, смежна с ней в дополнении $\bar{G}$. Когда очередь пуста, начинаем следующую компоненту с оставшейся вершины U .

Доказательство корректности. Алгоритм добавляет u в компоненту v только при наличии ребра $\{u, v\}$ в дополнении, поэтому разные истинные компоненты никогда не сливаются.

Пусть после завершения очереди осталась вершина u , принадлежащая той же компоненте дополнения. На кратчайшем пути дополнения от уже найденной части к u рассмотрим первое ребро, выходящее из найденной части. Когда его первый конец извлекался из очереди, второй конец находился в U и не был его соседом в G , поэтому алгоритм обязан был удалить его из U и добавить в очередь. Противоречие. Значит, каждая завершённая группа есть целая компонента. $\square$

Сложность. Удалённая из U вершина просматривается как непомеченная только один раз — это суммарно $\mathcal{O}(n)$. Если вершина u при просмотре списка остаётся, она помечена, то есть пара $\{u, v\}$ является ребром исходного графа; такое событие можно зарядить этому ребру, и каждое ребро заряжается не более двух раз. Маркировка списков смежности также суммарно $\mathcal{O}(m)$. Итого $\mathcal{O}(n + m)$ времени и $\mathcal{O}(n + m)$ памяти.